

PYTHON PROJECT

Employee Management System (EMS)

A hands-on Python project covering control structures, functions & object-oriented programming



PROJECT OVERVIEW

What is EMS?

A simplified Employee Management System built in Python to manage employee data through a console-based interface.

The system lets users add, view, and search employee records using a clean menu-driven approach.

Objective

- ✦ Practice control structures
- ✦ Write reusable functions
- ✦ Work with dictionaries & data
- ✦ Apply OOP concepts
- ✦ Build a real-world CLI app

TECHNOLOGIES & DATA STRUCTURE

Built With

Python 3

Dictionaries

Functions

Control Structures

OOP Concepts

CLI Interface

Data Structure

```
employees = {  
    101: {  
        'name': 'Alice',  
        'age': 27,  
        'department': 'HR',  
        'salary': 50000  
    }  
}
```

KEY FEATURES



Add Employee

Add new employees with ID, name, age, department & salary. Validates unique IDs automatically.



View All Employees

Displays all 100 pre-loaded employees in a neatly formatted table-style list.



Search Employee

Quickly search any employee by their unique Employee ID and view all their details.



Menu-Driven Interface

A continuously running menu loop keeps the app alive until the user exits gracefully.

CODE HIGHLIGHTS

`main_menu()`

```
def main_menu():  
    while True:  
        print('1. Add Employee')  
        print('2. View Employees')  
        print('3. Search Employee')  
        print('4. Exit')  
        choice = input('> ')  
        if choice == '1':  
            add_employee()  
        elif choice == '4':  
            break
```

`add_employee()`

```
def add_employee():  
    emp_id = int(input('ID: '))  
    while emp_id in employees:  
        print('ID exists!')  
        emp_id = int(input(...))  
    # collect other details  
    employees[emp_id] = {  
        'name': name,  
        'salary': salary  
    }
```

WHAT I LEARNED

01

Dictionary Mastery

Using nested dictionaries as a structured data store — CRUD operations on Python dicts.

02

Functions & Modularity

Breaking logic into reusable functions: `add_employee()`, `view_employees()`, `search_employee()`.

03

Control Flow

while loops, if/elif/else chains, and break statements to power the interactive menu.

04

Input Validation

Handling duplicate IDs gracefully with a validation loop before storing data.

05

Readable Code Structure

Organizing a multi-function project cleanly — a pattern that scales to larger OOP systems.



Thank You!

Open to feedback, collaborations & connections!

Built with Python • Submitted as part of my learning journey



*Connect with me on LinkedIn:
www.linkedin.com/in/shrutisree-ghosh-moulick*